SentinelOps An AI Assisted Desktop Control Plane for Automated DevOps Delivery

Subin and Praveen
Sathyabama Institute of Science and Technology Deemed to be University
Chennai India

Abstract—SentinelOps is a desktop prototype that turns an unfamiliar Git repository into a reviewed delivery plan. It clones a selected repository, identifies languages, frameworks, package managers, service boundaries, and likely start commands, then presents those findings for correction. The system validates the application locally before it generates delivery assets. Depending on the selected profile, SentinelOps prepares Docker and Compose files, CI definitions for GitHub Actions, GitLab CI, or Jenkins, Kubernetes manifests with an autoscaling policy, and observability material for Prometheus and Grafana. The design keeps installation and execution under user control: Docker remains an external prerequisite on Windows, and the application records commands, URLs, and health checks instead of hiding them. This paper describes the architecture, the module boundaries, and an evaluation plan for repository analysis and deployment readiness. The contribution is a practical, repo-aware control plane that makes the transition from source code to an inspectable DevOps workflow easier for student and small-team projects. It does not claim unattended production remediation or cloud deployment without explicit credentials and approval.

Index Terms—DevOps automation, repository analysis, CI/CD, containerization, observability, desktop application.

I INTRODUCTION

Many small software teams can write and run an application locally, yet still struggle to package it, choose a deployment target, create a continuous integration pipeline, and observe the running service. Those tasks require information that is scattered across package files, scripts, environment files, and repository layout. A developer who has not seen the project before may also miss that a React client and an API must start independently. Continuous delivery practices address this gap by treating build, test, and deployment work as part of software delivery rather than as a final manual step [2].

The reference work in [1] motivates the use of AI in DevOps decisions. SentinelOps narrows that idea to a desktop workflow that a user can inspect. It uses deterministic repository evidence first, then exposes an editable model of the detected stack. This choice is deliberate. A useful automation tool should show why it inferred a technology or command and should let the project owner correct it before it creates delivery files or starts containers.

II RELATED WORK AND MOTIVATION

Continuous delivery is commonly described as the ability to keep software in a releasable state through an automated build, test, and deployment process [2]. In practice, smaller projects often begin with only a source repository and a local start command. The gap between that starting point and a repeatable delivery process is larger than it appears. It includes dependency installation, environment configuration, packaging, test selection, image publishing, deployment selection, and operational visibility. Each step depends on details that may be implicit in the codebase.

Containerization addresses one part of this gap by packaging an application with its runtime dependencies. Docker's container model has been widely used to reduce differences between developer machines and deployment hosts [3]. However, a Dockerfile cannot be generated safely from a file extension alone. A front-end project may need a build stage and a static server, while an API may need a process manager, a database connection, and a health endpoint. SentinelOps treats the container template as a reviewed artifact rather than a final answer.

Observability provides the evidence needed after an application starts. Prometheus stores metrics as labeled time-series data and collects them from monitored targets [4]. Grafana can group queries into dashboards that help a user inspect the service state [6]. In a Kubernetes environment, an HPA adjusts replicas against observed resource or custom metrics [5]. These tools are useful only when their assumptions are visible. A dashboard with no reachable target and an autoscaler with no reliable metrics do not make a service safer.

Recent work on AI-driven DevOps focuses on decision support, anomaly detection, and automated remediation [1]. SentinelOps adopts the decision-support part of that direction. It uses a recommendation layer to suggest a stack or deployment profile, while the repository model, generated files, commands, and warnings remain inspectable. This makes the desktop workflow appropriate for project work where users must explain their design choices during demonstrations, reviews, or submissions.

The project therefore addresses a practical question: how can a desktop application take an unknown Git repository through a transparent path from inspection to locally validated delivery assets? The answer is not a single model or a single deployment target. It is a sequence of evidence-based modules with clear handoffs and a place for a user to correct uncertain information.

III DESIGN GOALS AND SCOPE

SentinelOps has five design goals. First, it must recognize common project shapes without assuming that every repository is a single service. Second, it must separate local verification from deployment generation so a broken application is visible early. Third, it must produce ordinary, editable artifacts rather than hiding configuration behind a graphical interface. Fourth, it must keep local machine changes limited: Docker Desktop is checked as an external prerequisite on Windows and is not installed by the application. Finally, it must make operational choices visible through a deployment profile, health checks, scaling settings, and an observability plan.

The prototype targets Node.js, React, Next.js, Vue, Angular, Python, Flask, FastAPI, Django, Go, and Rust projects. Its detector looks for package manifests, lock files, framework dependencies, source layout, scripts, ports, and common container configuration. The output is a proposed model, not a guarantee. Users can edit the detected language, framework, service type, runtime commands, ports, and selected CI provider before proceeding.

IV SYSTEM ARCHITECTURE

Figure 1 summarizes the controlled flow. Repository intake produces a local workspace and Git metadata. The analysis module builds an evidence-backed project model. Local verification then installs declared dependencies, starts the service using the selected command, probes a health endpoint when one is available, and records URLs that a user can open in a browser. Only after that gate does the asset generator prepare container, pipeline, deployment, and monitoring files.

Fig. 1. SentinelOps controlled delivery flow.

Repository intake	Local validation	Delivery assets

and stack analysis	and service URLs	and operations plan

The orchestration layer does not replace a production platform. It coordinates choices made by the user. For example, a local profile can favor a native process for early testing, while a container profile can generate Compose assets for a reproducible local environment. A Kubernetes profile adds a Deployment, Service, and HorizontalPodAutoscaler manifest. Kubernetes describes the HPA as a controller that adjusts workload replicas according to observed metrics [5], so SentinelOps treats scaling thresholds as policy that must be reviewed, not as an unexplained automatic action.

Each project follows a state model. The initial state contains a repository address and a project name. After cloning, the repository model contains metadata such as the remote URL, current branch, commit identifier, file inventory, and detected components. The review state contains user-selected commands, ports, and a target profile. The validated state adds command output, a health result, and local URLs. The generated state records the paths of Docker, pipeline, deployment, and monitoring files. Storing these transitions gives the user a concise history of how a delivery decision was reached.

The desktop interface is organized around these states instead of forcing users through a hidden linear wizard. A repository can be re-analyzed after a correction, and a generated plan can be changed without recloning the project. This is important when a repository uses a nonstandard script name, a custom port, or a monorepo layout. The user should be able to amend the model at the point where the assumption is wrong rather than edit a downstream file and lose the connection to the original evidence.

The architecture also separates private information from generated configuration. Source URLs and local paths may be stored in the project record, while registry credentials, deployment secrets, and remote cluster tokens are never guessed by the detector. If a later integration needs these values, it should request them through the user interface and pass them through a secure host-specific storage mechanism. This separation reduces the chance that generated examples encourage users to commit secrets to a repository.

V IMPLEMENTATION DETAILS

The desktop application is built around a narrow set of modules. The Git module clones a repository into a project-specific directory and supports deletion of that cloned workspace after confirmation. The detector reads repository files and calculates findings from observable evidence. It flags client and server directories separately when it finds independent package manifests or framework signatures. This allows the interface to display two start commands and two local addresses when a project needs them.

The validation module resolves the selected install command, starts the application with the selected runtime command, and captures output for the user. It reports the process state and the local URL instead of claiming that the application is ready solely because a command was launched. A successful probe is useful evidence, but it is still scoped to the chosen endpoint and local environment. This distinction prevents generated deployment assets from being confused with a verified production release.

For containerized execution, SentinelOps derives a Dockerfile template from the confirmed runtime. Docker packages an application with the libraries and configuration it needs, which helps reduce environment differences across development and deployment [3]. The generated file remains editable because build steps, secrets, multi-stage builds, and operating-system packages are project-specific. Docker image pulls and container runs are available only after the host reports that Docker is installed and usable.

The pipeline module selects GitHub Actions, GitLab CI, or Jenkins based on repository context and user choice. It generates a visible sequence for checkout, dependency installation, build, test, image build, and optional publish. Credentials, registry URLs, and production deployment commands are intentionally not inferred. They require values supplied by the user or organization.

Repository analysis begins with files that carry explicit project intent. For JavaScript and TypeScript projects, the detector examines package.json, package-manager lock files, declared scripts, and framework dependencies. For Python projects, it examines requirements files, pyproject metadata, common entry files, and framework imports. Comparable rules identify Go modules, Rust manifests, and existing container files. The detector then combines file evidence with directory evidence. A client folder with its own manifest and a server folder with a different manifest is reported as a split application rather than as one ambiguous project.

The recommended commands are derived from the inspected manifests. For example, a declared development script has stronger evidence than a generic framework default. SentinelOps records the source of a command in the project model, so the interface can distinguish a direct repository script from a fallback suggestion. This is useful when the user changes a command manually. The revised command becomes the selected value for local validation and later asset generation, while the original detection remains visible for comparison.

Local validation is intentionally separate from Docker execution. The application first checks whether the selected runtime is available, runs the dependency installation command in the cloned workspace, and starts the service with a managed process. Output is streamed into the desktop interface and is retained with the validation record. Where a web address can be inferred, SentinelOps probes the endpoint and displays a clickable local URL. For a project with separate client and server services, the result includes independent process states and URLs instead of a single generic success message.

The Docker service uses a host status check before it offers image-related actions. On Windows, SentinelOps tells the user that Docker Desktop must be installed and started; it does not attempt to install it or modify the host. When Docker is available, the project can pull a selected base image, build an application image, and run a container with the proposed port mapping. The generated Dockerfile and Compose file are shown as normal text assets so that the user can change base images, build arguments, environment variables, volumes, or service names before execution.

A free language model can be used as a secondary recommender for a stack, delivery path, or explanation of an error. The recommendation layer is bounded by the repository model. It should not invent a framework that the detector did not observe, execute shell commands without approval, or overwrite generated files without preserving the previous version. In this design, model output is treated as a proposal accompanied by evidence and confidence notes. Deterministic checks remain responsible for file discovery, command execution, and validation status.

The project includes a visualization view that presents the chosen technology stack and the planned delivery path. It maps repository findings to the selected CI provider, container base image, deployment profile, monitoring assets, and response policy. The purpose is operational clarity. A user can see, for example, that a FastAPI service was paired with Python, Docker, a GitHub Actions workflow, a Kubernetes profile, Prometheus scraping, and

a CPU-based HPA before any remote deployment command is considered.

Error handling is designed around recovery rather than silent retry. A clone error keeps the project in the intake state and surfaces the Git message. A detection error preserves the repository record and asks the user to select fields manually. A failed install or local start stores the terminal output and prevents the interface from presenting deployment generation as a verified path. A Docker error reports the missing host prerequisite or engine result. These boundaries keep a failure trace close to the action that caused it.

TABLE I. SENTINELOPS MODULES AND GENERATED OUTPUTS

Module	Output or decision
Repository intake	Cloned workspace, commit metadata, and project inventory
Stack analysis	Editable language, framework, package, and frontend/backend findings
Verification	Install and start commands, health result, and local access URLs
Delivery assets	Dockerfile, Compose file, CI workflow, Jenkinsfile, or GitLab pipeline
Operations	Deployment profile, HPA manifest, Prometheus configuration, dashboard JSON, and runbook

VI USER WORKFLOW AND CONTROLLED EXECUTION

A user begins by creating a SentinelOps project and supplying a Git repository address. The application records a human-readable project name and chooses a local directory for the clone. If the repository already exists in a previous project workspace, the user can inspect its metadata or remove the cloned copy through a dedicated delete action. Deletion applies only to the copied workspace managed by SentinelOps; it must not remove a user's original repository or any remote repository. This small constraint matters because users often experiment with several repositories while learning the tool.

After cloning, the analysis screen presents the detected stack as editable fields rather than as a black-box result. It can list a primary language, framework, package manager, build tool, service type, detected ports, container support, and CI configuration. When the repository contains both a front end and a back end, the screen shows each component separately. A user can confirm a React client on one port and a Python API on another, then keep those selections when creating Docker or pipeline assets.

The next step is local validation. The user reviews the install and start commands before running them. SentinelOps then displays terminal output, process status, and local addresses. This workflow is especially useful for a project whose front end and back end must be launched separately. Instead of asking the user to infer the address from logs, the application shows an explicit link such as a client URL and an API URL when they are known. If a process stops unexpectedly, its exit result becomes part of the validation record.

Once validation succeeds or the user elects to continue with a known limitation, the planning screen recommends a delivery profile. The recommendation considers the selected stack, whether the service is split, whether a Dockerfile already exists, and the user's stated target. The screen explains its recommendation in ordinary language. For example, it may suggest native local execution for an early single-service prototype, Compose for a service with a local dependency, or Kubernetes only after a container image and cluster target have been confirmed.

The execution screen retains a visible distinction between generated files and commands that have actually run. Creating a Dockerfile does not mean an image exists. Generating a Jenkinsfile does not mean a pipeline has been configured on a Jenkins server. Preparing a Prometheus configuration does not mean metrics are being collected. This distinction prevents an attractive visualization from overstating system state. Every stage can display a generated, configured, started, validated, failed, or not-applicable status.

The final project view acts as a compact record for demonstrations. It can show the repository source, selected stack, local test result, container choice, CI/CD template, deployment profile, observability configuration, scaling policy, and recommended maintenance actions. The same record gives a reviewer enough context to understand why the application chose its assets. It also creates a useful handoff note if another developer needs to continue the setup without repeating the initial repository investigation.

VII DELIVERY PROFILES AND CI CD

SentinelOps presents deployment as a choice among profiles rather than a one-size-fits-all action. The native local profile is the least complex option. It starts the selected command directly on the workstation and is useful for debugging a newly cloned repository. The Docker Compose profile packages the application and supporting services in a repeatable local bundle. The Kubernetes profile is intended for users who already have an approved cluster and understand the operational implications of a remote deployment. Table III summarizes the distinction.

TABLE III DEPLOYMENT PROFILES EXPOSED BY SENTINELOPS

Profile	Appropriate use	Generated guidance
Native local	Early debugging on one workstation	Commands, URLs, health status, stop action
Docker Compose	Repeatable local service bundle	Dockerfile, Compose file, image and port mapping
Kubernetes	Cluster deployment with approval	Deployment, Service, HPA, capacity policy

The recommended profile depends on the confirmed project model. A small static front end can often be tested locally and then built into a compact web-serving image. A service that requires a database may benefit from Compose during development because the app and its dependency can be described together. A split front end and back end needs separate service entries, port mappings, and health checks. Kubernetes becomes a reasonable recommendation only when a cluster, image registry, and operational ownership are already available. SentinelOps states these prerequisites in the profile description instead of presenting Kubernetes as a default upgrade.

The CI/CD templates have the same principle. A GitHub repository can begin with GitHub Actions, a GitLab repository with GitLab CI, and a Jenkins-based environment with a Jenkinsfile. Each template starts with checkout and dependency installation, then runs the project build and selected tests. Container build and push steps remain optional because an image registry requires a user-supplied destination and credentials. The template can also generate a deployment stage, but it remains disabled until the user supplies the target environment and approval process.

The generated pipelines are designed to be read by a developer who did not build SentinelOps. Steps use clear names and standard repository scripts where possible. A pipeline should fail when a build or test fails, and the report should point back to the command output. The desktop application does not promise to repair every failure automatically. It can identify a failed stage, collect the message, explain common causes, and propose a next check. This is safer than retrying a build indefinitely or modifying source code without review.

Image tagging is treated as a release decision. A generated configuration can use a local tag for testing and can reserve immutable tags based on a commit identifier for an approved registry workflow. A later production integration should add provenance, vulnerability scanning, and access control before an image is promoted. SentinelOps can prepare the surrounding files, but the project owner remains responsible for registry permissions and the policy that authorizes an image to reach an environment.

VIII OBSERVABILITY AND MAINTENANCE

Prometheus collects time-series metrics from configured targets, while Grafana provides dashboards that make those metrics easier to inspect [4], [6]. SentinelOps creates a Prometheus scrape configuration, a starter Grafana dashboard definition, and an optional Compose stack for local observation. The desktop interface can summarize generated endpoints and recommended signals such as service availability, CPU use, memory use, request errors, and restart count. It must not claim live metrics until the user has started the monitoring stack and the application exposes a compatible metrics endpoint.

The maintenance model is recommendation-led. When a health probe fails or an observed threshold is exceeded, the interface can propose a restart, rollback, scaling change, configuration review, or ticket. A runbook explains the suggested action and the evidence that triggered it. Automatic source-code repair and unapproved production changes are outside the prototype scope. This boundary makes an automated workflow more suitable for a desktop tool used by learners and small teams, where an incorrect assumption can otherwise change a local project or remote environment unexpectedly.

The observability configuration starts with a small, explicit target list. The user selects the application endpoint and the scrape interval, and SentinelOps writes a Prometheus configuration that can be reviewed before it is started. A starter Grafana dashboard groups availability, CPU, memory, process restarts, and request-error panels when the chosen runtime exposes those metrics. The dashboard definition is a starting point, not proof that every application has the same metric names. A Python API, a Node.js process, and a static client expose different signals and may require an instrumentation change before the panels show data.

Autoscaling is available only for the Kubernetes profile. SentinelOps generates an autoscaling v2 resource with reviewed minimum and maximum replica values and a target CPU utilization. It also includes resource requests in the generated workload because CPU-based scaling relies on those requests. The user can adjust the capacity policy in the interface and see the resulting manifest. The system explains that high traffic alone does not guarantee a scale-out event; the configured metric, target, workload readiness, and cluster capacity all affect the result.

A controlled self-healing loop uses four stages. First, a health or metric signal identifies an abnormal condition. Second, the system gathers context such as the selected profile, recent command output, configured endpoint, and the last known status. Third, it matches the event to a runbook suggestion. Finally, it requests user approval for any action that changes a running environment. This can include a local restart, a rollback to a previously selected configuration, a scaling adjustment, or a notification. The loop is useful because it makes the suggested response explainable and reversible.

Notifications are an integration boundary rather than an assumed service. The project model can reserve hooks for email, Slack, Microsoft Teams, Jira, or PagerDuty. A real connection requires credentials, endpoint configuration, and an organization-specific escalation policy. In the current project, the user-facing value is the event record and recommended action. This keeps the paper aligned with what SentinelOps can safely generate and avoids claiming that an alert was delivered when no integration has been configured.

The desktop visualization supports the same maintenance model. A generated plan can show the container image, CI/CD stages, deployment profile, metric sources, autoscaling range, and recent health result as connected modules. The visual view does not replace Grafana. Instead, it explains how the generated assets relate before the user opens a monitoring dashboard. That distinction is helpful during project demonstrations because viewers can follow the path from repository analysis to an operational policy without needing live cloud credentials.

IX EVALUATION PLAN

The prototype should be evaluated with a curated set of public repositories that cover single-service Node.js and Python applications, front-end applications, and repositories with separate client and server folders. For each repository, an evaluator records the expected language, framework, package manager, start command, ports, and service boundaries before running SentinelOps. The primary measures are detection precision, detection recall, the rate of correct local launch instructions, and the number of generated artifacts that require manual correction.

A second evaluation should examine operational transparency. Participants should be able to identify the selected deployment profile, the command used for local validation, the container image tag, the metrics endpoint, and the condition that would request scaling. A short user study can compare this workflow with a manual checklist. The purpose is to find missing evidence and confusing defaults, not to make unsupported claims about deployment speed or reliability.

TABLE II PROPOSED REPOSITORY EVALUATION CASES

Project shape	Expected finding	Evidence reviewed
Single API	Runtime and port	Manifest, scripts, source entry point
Web client	Framework and build command	Dependencies, lock file, public assets
Client plus server	Two services and URLs	Independent manifests and folders
Container-ready repo	Existing files preserved	Docker and Compose configuration

The evaluation can also score the quality of manual correction. An evaluator deliberately changes a command, a port, or a framework field and then checks whether the regenerated Dockerfile, pipeline, and deployment artifacts reflect the selected value. A good result is not merely a high automatic detection rate. It is a tool that remains coherent after a user corrects the detector. This matters because repositories often contain legacy scripts or examples that should not be chosen as a production entry point.

Failure-oriented tests are equally important. The test set should include an invalid Git URL, a private repository without access, a dependency-install failure, a missing runtime, a server that starts but does not answer the health probe, a missing Docker engine, and a missing metrics endpoint. For each case, the evaluator records whether SentinelOps preserves the project state, exposes the error source, avoids a misleading ready status, and offers a useful next action. These tests assess reliability of the workflow itself rather than performance of a particular application.

The paper does not report numeric outcomes because a full benchmark set and participant study have not yet been completed. That restraint is important. The prototype demonstrates the architecture and generated artifacts, while the proposed study defines how its accuracy and usability can later be measured. Results should include the repository sample, exact versions,

selected manual overrides, and failures so that another student team can reproduce the comparison.

X SECURITY LIMITATIONS AND FUTURE WORK

Repository inspection cannot reliably discover every runtime requirement. Private registries, undocumented environment variables, external databases, and shell scripts may require manual input. Dynamic stack selection also depends on the quality of the underlying evidence and any optional language model recommendation. SentinelOps therefore records both its detection and the user's final selections. Future work can add signed artifact provenance, secret scanning, configurable policy checks, richer service discovery, and a controlled approval workflow for remote deployment.

Security requires further work before SentinelOps is used outside a controlled development setting. Generated containers should run with least privilege, use small maintained base images, and avoid embedding secrets. CI/CD credentials should be stored in the provider's secret mechanism rather than copied into generated YAML files. A production deployment path should include access review, image scanning, signed releases, and a rollback plan. These controls are intentionally described as requirements instead of being presented as completed capabilities.

The current scope also excludes autonomous code changes. A language model can help interpret logs or describe a likely configuration issue, but it may produce a plausible yet incorrect fix. SentinelOps therefore limits automated behavior to inspection, generation, local process control, health observation, and recommendations that a user can review. Future versions could add a sandboxed repair branch, a test-only validation gate, and a pull-request workflow for proposed changes. That approach would retain human review while making recovery experiments more systematic.

Another future direction is richer architecture discovery. A repository may contain infrastructure modules, database migrations, message queues, mobile clients, or multiple deployable services. Extending the project model to represent those components explicitly would improve deployment recommendations. The same model could support cost-aware choices, environment-specific configuration, and dependency graphs. Such enhancements should continue to preserve the current principle that generated conclusions are traceable to files, user selections, or stated operational policies.

XI DEMONSTRATION SCENARIO

A representative SentinelOps session begins with a repository that contains a React client and a Python API. The user creates a project, pastes the repository URL, and starts the clone. The Git module displays the remote address and local workspace path after the operation completes. The detector then finds two independent package descriptions and reports a client component and a server component. It proposes a JavaScript front-end framework, a Python API framework, separate dependency commands, and two ports. The user can revise any of these values before they become part of the project plan.

The user next runs local validation. SentinelOps installs the front-end and back-end dependencies in their respective folders, starts each service, and streams the output into the workspace view. The result reports the front-end URL and the API URL separately. If the front end can load while an API route fails, the record shows that difference instead of treating the project as fully healthy. A developer can use the captured output to change an environment variable, port, or start command and then repeat the test. This scenario demonstrates why separate service discovery is useful for full-stack repositories.

After a successful local check, the planner can recommend Docker Compose as the initial deployment profile. It prepares one service definition for the client and one for the API, with their chosen ports and a shared network. The user reviews the proposed Dockerfiles, chooses a supported base image, and starts image pull or build actions only if Docker is ready on the host. The Docker status view makes the prerequisite visible on Windows. It does not offer a hidden installation path or claim that a container was built before the engine returns a result.

For source delivery, the user selects the CI provider that matches the repository host. A GitHub repository can receive a GitHub Actions workflow with checkout, dependency installation, build, test, and optional image build stages. A GitLab repository can receive a comparable GitLab CI definition. A Jenkins environment can receive a Jenkinsfile that its administrator may import into an existing server. The template remains a generated file in the project workspace. Connecting it to a remote service is a later deliberate action that needs credentials and repository permissions.

If the user later chooses a Kubernetes profile, SentinelOps converts the selected service information into editable workload manifests. The plan can include a Deployment, Service, resource requests, and an HPA with an explicit replica range. The interface explains that this profile requires a cluster, an image registry, and access approval. It also makes clear that the HPA uses the selected resource policy. It does not imply that traffic automatically produces a cloud deployment or that the desktop app can scale a cluster it cannot access.

The observability step adds a local Prometheus configuration and a starter Grafana dashboard definition. Once the user starts the monitoring services and exposes application metrics, the plan can connect the application target to dashboard panels. A failed scrape is shown as a missing dependency or configuration issue, not as a successful observation. The maintenance view can then associate a failed health check or high utilization signal with a runbook entry. The user sees the evidence, suggested action, and whether approval is required before any corrective operation.

This scenario illustrates the intended meaning of automation in SentinelOps. The desktop application removes repetitive discovery and configuration work, but it does not hide decisions that carry operational risk. A student can demonstrate the entire flow from repository to generated artifacts on one machine. A small team can use the same flow to create a consistent starting point before involving a registry, CI server, cluster administrator, or production monitoring environment.

XII REPRODUCIBILITY AND MAINTAINABILITY

The project favors text-based outputs because they are easy to inspect, diff, and version with the source code. Dockerfiles, Compose files, CI pipelines, Kubernetes manifests, Prometheus configurations, Grafana dashboard JSON, and runbooks should be stored in a predictable project folder. SentinelOps records the source repository revision used for analysis and the selected settings used for generation. If a user regenerates an asset after changing the stack model, the application can compare the new configuration with the earlier version and show which assumptions changed.

A maintainable tool also needs predictable cleanup. SentinelOps gives each cloned repository its own project workspace and provides a specific option to delete that cloned copy. It should stop managed local processes before removing the workspace and should clearly name the target directory before confirmation. Generated assets can be kept even when the clone is removed if the

user wants to retain a design record. This behavior lets users recover disk space without losing the history of their chosen deployment plan.

For academic use, repeatability depends on documenting the host assumptions. A project demonstration should state the operating system, runtime versions, Docker availability, repository commit, selected commands, and chosen profile. Screenshots or exported project summaries can capture the detected stack and generated flow. These details make it possible for another evaluator to distinguish a product limitation from a missing host prerequisite, and they make the proposed evaluation in Section IX more credible.

XIII CONCLUSION

SentinelOps provides a practical path from a Git repository to a reviewed DevOps plan. Its central idea is simple: inspect the project, prove that it can run locally, then generate transparent assets for containers, CI/CD, deployment, and monitoring. The prototype keeps users in control of uncertain decisions and clearly separates generated files from verified production operations. That combination makes the system appropriate for learning, project demonstrations, and small software teams that need a repeatable starting point for delivery automation.

REFERENCES

[1] K. Raj, K. P. Poojary, Abhay, A. R. S., and L. K. S. R., “AI Driven DevOps for Intelligent Automation in Continuous Software Delivery Pipelines,” in Proc. 3rd Int. Conf. on Emerging Computing Systems and Network, 2025.

[2] J. Humble and D. Farley, Continuous Delivery Reliable Software Releases through Build Test and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.

[3] D. Merkel, “Docker Lightweight Linux Containers for Consistent Development and Deployment,” Linux Journal, vol. 2014, no. 239, 2014.

[4] Prometheus Authors, “Overview,” Prometheus Documentation. [Online]. Available: https://prometheus.io/docs/introduction/overview/. Accessed: Sep. 24, 2026.

[5] The Kubernetes Authors, “Horizontal Pod Autoscaling,” Kubernetes Documentation. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/. Accessed: Sep. 24, 2026.

[6] Grafana Labs, “Build Your First Dashboard,” Grafana Documentation. [Online]. Available: https://grafana.com/docs/grafana/latest/fundamentals/getting-started/first-dashboards/. Accessed: Sep. 24, 2026.